

Lab4-LOG430 – Test de charge et Observabilite

 CI Pipeline - Lab4 passing

Dans ce laboratoire j'ai effectué la correction des remarques du professeur Fabio et implémenter les fonctionnalités correctement ! Vous pouvez essayer toute les nouvelles fonctionnalités :

<http://10.194.32.174:5000/login>

1. Architecture du projet

Le projet suit :

- **app.py** :
 - Role : Point d'entrée Flask
 - enregistre les blueprints, configure Swagger, etc.
 - Initialise l'application Flask,
 - Configure Swagger (documentation API),
 - Enregistre les différents blueprints (web pour l'interface web, api pour l'API REST),
 - Peut gérer la configuration globale, le CORS, les middlewares globaux, etc.
- **routes.py** : Routes API REST, Blueprint nommé api
 - Role : Définit les routes REST API, utilisées par des clients externes ou du JavaScript front-end
 - Utilise un Blueprint nommé api.
 - Toutes les routes sont en /api/... (ex : /api/products, /api/ventes, etc.).
 - Réponses toujours en JSON.
 - Sécurisation possible via token.
 - Contient des docstrings Swagger pour la documentation automatique.
- **web_routes.py** : Routes classiques pour le web (login, gestion HTML, ...), Blueprint nommé web
 - Role : Définit les routes classiques web (HTML), destinées à être utilisées par les utilisateurs via le navigateur.
 - Utilise un Blueprint nommé web.
 - Gère la connexion, l'affichage des pages HTML, les formulaires, etc
 - Retourne des templates HTML (ex : render_template("index.html", ...)).
 - Peut utiliser la session Flask pour l'authentification utilisateur.
 - Routage classique (ex : /login, /logout, /magasins, /products...).
- **src/db** : db.py / init_db.py
 - Role : Gestion de la base de données SQLAlchemy.
 - **db.py**: Définit la session, la connexion, le moteur, etc.
 - **init_db.py**: Script d'initialisation (création des tables, population de données...).
- **src/models/** : models.py, product.py, sale.py, store.py, etc.
 - Role : Définition des classes modèles ORM (SQLAlchemy) pour représenter les entités de la base (Product, Store, Sale, etc.).
- **src/services/** : product_service.py, sale_service.py, store_service.py, etc.
 - Role : Contient la logique métier (business logic) et les fonctions/services manipulant les entités (ex : création d'un produit, vente, remboursement, etc.). Elle sert d'interface entre les routes et les modèles. Elle permet de garder les routes propres et de factoriser le code métier.

- **src/templates/** : Fichiers HTML (Jinja2). Ce sont tous tes templates pour l'affichage côté utilisateur.

Voici la difference entre les deux routes : web_routes.py permet aux utilisateurs d'utiliser le système dans un navigateur web. Mais routes.py sert à exposer les données et opérations pour des machines ou autres systèmes. Ainsi, le but principal de routes.py = rendre ton backend réutilisable et ouvert

Pour accéder au metrics brut : <http://10.194.32.174:5000/metrics>

2. Test de charge initial avec K6

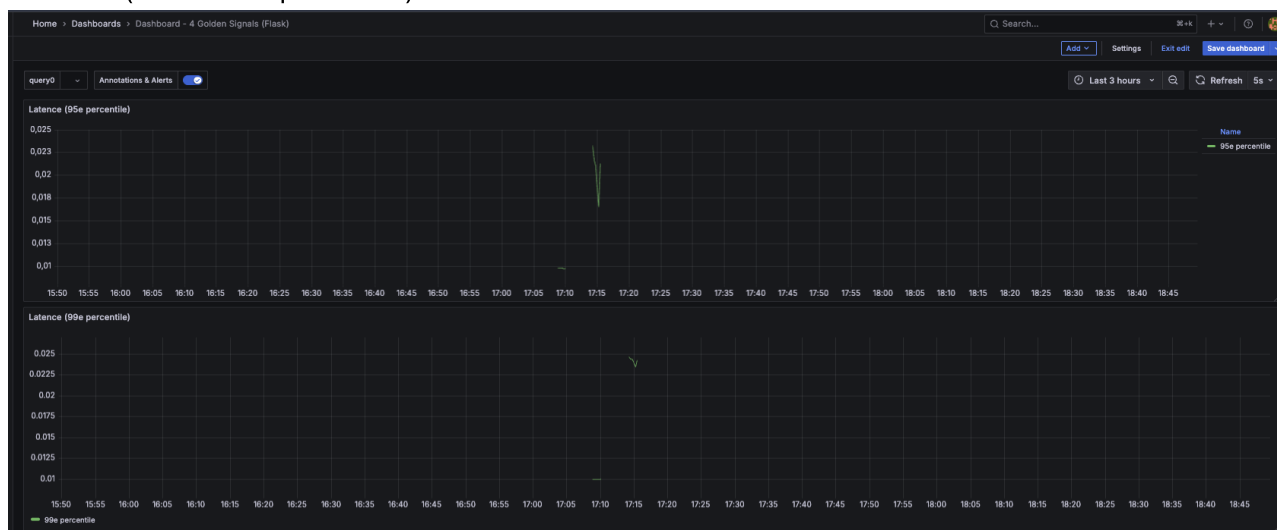
Un test de charge est réalisé avec l'outil K6 pour simuler une montée en charge et observer les métriques suivantes : les 4 Golden Signals

- Latence (p95 / p99) : temps de reponse moyen
- Trafic/Requêtes par seconde
- Saturation : utilisation charge CPU / Mémoire, threads, pool de connexions.
- Erreurs : taux de réponses HTTP 4xx ou 5xx.

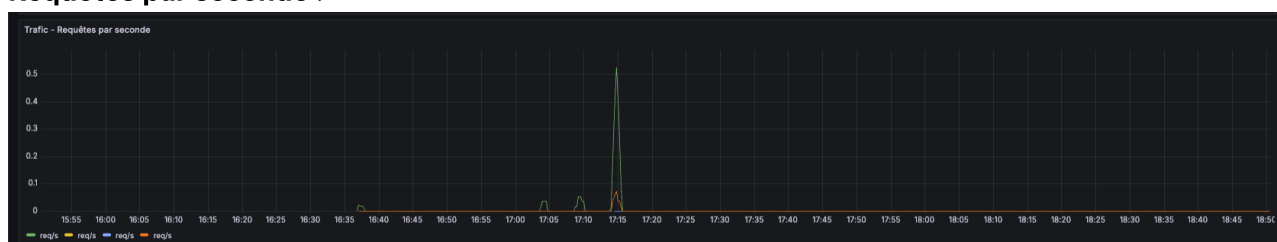
🔍 Tableau de bord Grafana

Des dashboards Grafana sont utilisés pour visualiser les résultats du test :

- **Latence** (95e et 99e percentile) :



- **Requêtes par seconde** :



- **Utilisation CPU & RAM :**



- **Fichiers ouverts :**



Les graphiques qui ne présentent pas de data c'est parcequ'il n'y a pas de données d'erreur.

Explication des axes sur les graphes Grafana :

- **Axe des abscisses (horizontal) :** Temps (en heures:minutes)
- **Axe des ordonnées (vertical) :**
 - Pour la latence : temps de réponse en secondes
 - Pour la charge CPU : pourcentage d'utilisation (de 0 à 1 = 0% à 100%)
 - Pour la mémoire : en octets
 - Pour les requêtes par seconde : nombre de requêtes traitées par seconde

3. Résultats des tests de charge (K6 + Grafana)

Scénario 1 — Infrastructure de base (sans cache, sans load balancer)

L'objectif c'est d'évaluer les performances de l'application dans sa version initiale.

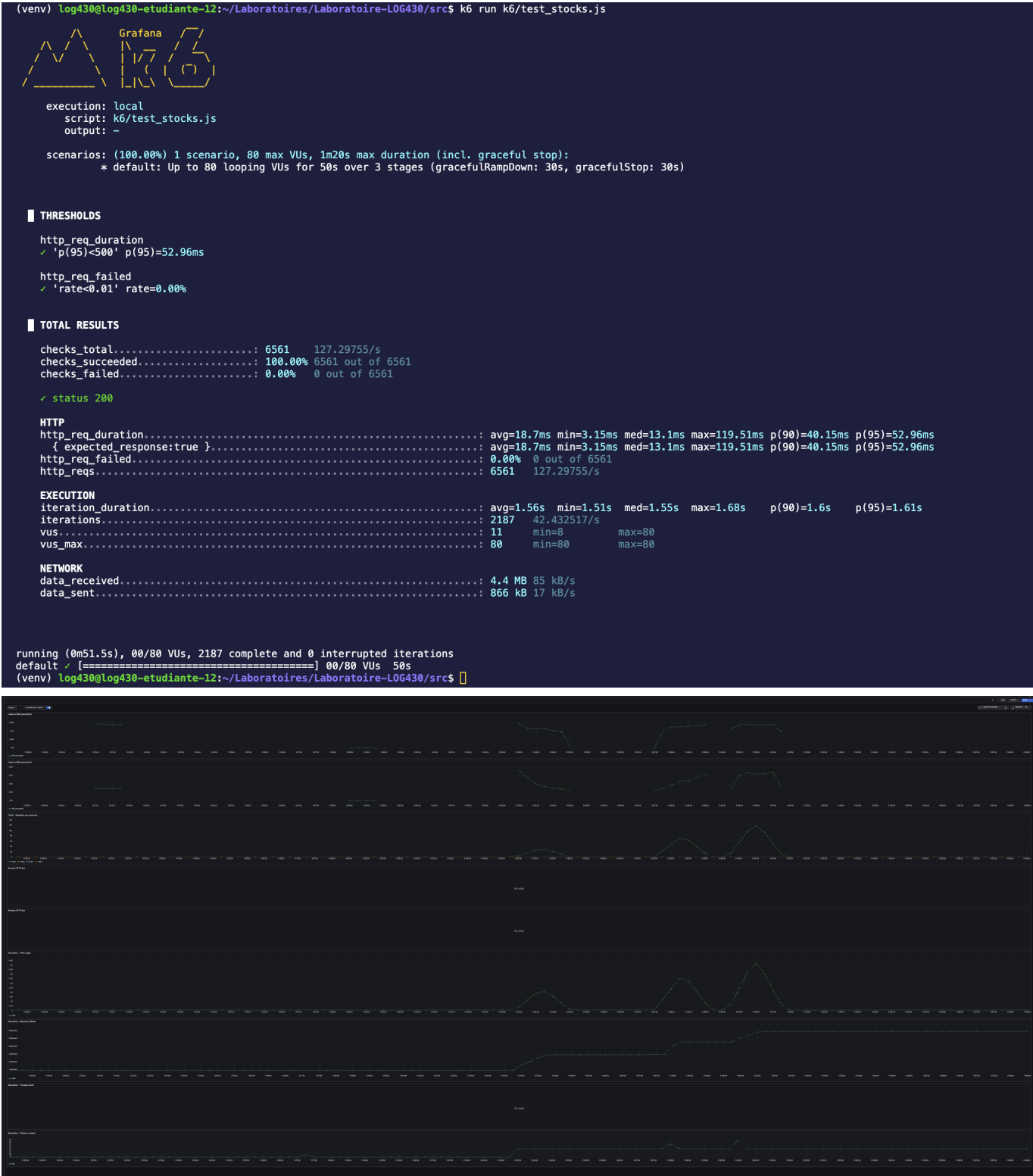
A: Consultation simultanée des stocks

Ce test simule une charge générée par 80 utilisateurs virtuels (VUs) accédant simultanément aux stocks de trois magasins via l'API REST (GET /api/magasins/:id), avec authentification Bearer. Le fichier de test est dans le dossier `src/k6/test_stocks.js`

Voici les conditions :

p(95)<500 : 95% des requêtes doivent répondre en moins de 500 ms.

rate<0.01 : Moins de 1% d'échecs tolérés.



- ✓ Résultats observés :
- Total des requêtes HTTP : 6561
 - Taux de succès : 100% (6561/6561)
 - Durée moyenne de requête : ~18.7 ms (p95 = 52.96 ms)
 - Durée moyenne d'une itération : ~1.56 s
 - Aucun échec constaté (http_req_failed = 0.00%)

B: Génération de rapports consolidés

L'objectif est de tester la robustesse du serveur avec jusqu'à 500 utilisateurs simultanés accédant à un rapport. Le fichier de test est dans le dossier `src/k6/test_reports.js`

Résultats :

- ❌ 10510 requêtes, dont 1.86% échouées (soit 196 erreurs)
- ❌ 98 requêtes n'ont pas renvoyé de status 200
- ❌ Check rapport contient données échoué dans 98 cas (données manquantes ou incorrectes)
- ❌ Seuil p(95)<500ms non respecté :
 - Temps de réponse p(95) : ~12.1s
 - Max : ~52.8s
- ❌ Le système a montré des signes de saturation au-delà de 4 VUs effectifs (malgré la cible de 500)

Conclusion : Le endpoint `/api/rapport` ne tient pas la charge à grande échelle (≥ 500 VUs). Il nécessite : une optimisation backend (base de données, logique métier)

Voici les résultats à Faible charge :

```
(venv) log430@log430-etudiante-12:~/Laboratoires/Laboratoire-LOG430/src$ k6 run k6/test_reports.js

      Grafana
    /  /  /  /
   /  /  /  /
  /  /  /  /
 /  /  /  /
/  /  /  /

execution: local
script: k6/test_reports.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 1m20s max duration (incl. graceful stop):
  * default: Up to 10 looping VUs for 50s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

■ TOTAL RESULTS

checks_total.....: 802      15.742905/s
checks_succeeded.....: 100.00% 802 out of 802
checks_failed.....: 0.00% 0 out of 802

✓ status 200
✓ rapport contient données

HTTP
http_req_duration.....: avg=17.02ms min=8.93ms med=16.58ms max=87.14ms p(90)=19.52ms p(95)=21.16ms
  { expected_response:true }.....: avg=17.02ms min=8.93ms med=16.58ms max=87.14ms p(90)=19.52ms p(95)=21.16ms
http_req_failed.....: 0.00% 0 out of 401
http_reqs.....: 401      7.871452/s

EXECUTION
iteration_duration.....: avg=1.01s min=1.01s med=1.01s max=1.08s p(90)=1.02s p(95)=1.02s
iterations.....: 401      7.871452/s
vus.....: 1      min=1      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 427 kB 8.4 kB/s
data_sent.....: 52 kB 1.0 kB/s

running (0m50.9s), 00/10 VUs, 401 complete and 0 interrupted iterations
default ✓ [=====] 00/10 VUs 50s
(venv) log430@log430-etudiante-12:~/Laboratoires/Laboratoire-LOG430/src$ git pull
```

Voici les resultat à forte charge :



C: Mise à jour de produits à forte fréquence

L'objectif est de mettre à jour un produit (productId = 1) de manière concurrente en simulant jusqu'à 500 utilisateurs virtuels (vus) durant 30 secondes

On a effectué 2 tests avec des charges utilisateurs différents et bien évidemment comme avec les autres API des qu'on passe au dessus de 50 users l'application Python n'est plus capable de répondre à 100% des requêtes.

✓ Test 1 : Charge modérée (40 VUs max)

- Nombre total de requêtes : 1200
- Taux de succès : 100%
- Durée moyenne des requêtes : 24.5 ms
- Aucune erreur HTTP détectée

```
(venv) log430@log430-etudiante-12:~/Laboratoires/Laboratoire-LOG430/src$ k6 run k6/test_updates.js
```



```

execution: local
script: k6/test_updates.js
output: -

scenarios: (100.00%) 1 scenario, 40 max VUs, 1m0s max duration (incl. graceful stop):
* default: 40 looping VUs for 30s (gracefulStop: 30s)

■ TOTAL RESULTS

checks_total.....: 1200    38.73125/s
checks_succeeded.....: 100.00% 1200 out of 1200
checks_failed.....: 0.00% 0 out of 1200

✓ update status 200

HTTP
http_req_duration.....: avg=24.5ms min=3.07ms med=14.43ms max=270.14ms p(90)=46.23ms p(95)=72.13ms
{ expected_response:true }.....: avg=24.5ms min=3.07ms med=14.43ms max=270.14ms p(90)=46.23ms p(95)=72.13ms
http_req_failed.....: 0.00% 0 out of 1200
http_reqs.....: 1200    38.73125/s

EXECUTION
iteration_duration.....: avg=1.02s min=1s med=1.01s max=1.27s p(90)=1.04s p(95)=1.07s
iterations.....: 1200    38.73125/s
vus.....: 40 min=40 max=40
vus_max.....: 40 min=40 max=40

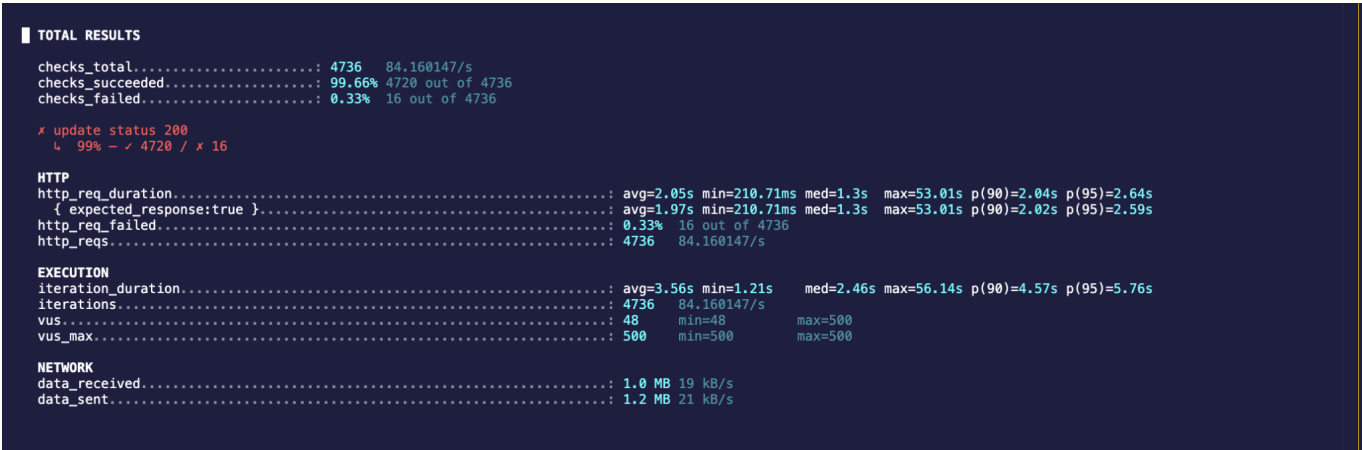
NETWORK
data_received.....: 264 kB 8.5 kB/s
data_sent.....: 305 kB 9.8 kB/s

```

Le serveur a parfaitement géré la charge, avec un temps de réponse stable et rapide.

△ Test 2 : Charge élevée (500 VUs)

- Nombre total de requêtes : 4736
- Taux de succès : 99.66%
- Taux d'échec : 0.33% (16 erreurs)
- Durée moyenne des requêtes HTTP : ~2s
- Pire temps de réponse : 53.01s
- Durée d'exécution moyenne : 3.56s, avec un pic à 56.14s



Cela indique une limite d'échelle au-delà de laquelle une mise en cache, un load balancing ou une optimisation du backend serait nécessaire.

Ainsi c'est pour cela que nous allons passer au scénario 2.

Scénario 2 — Ajout du Load Balancer

Load Balancing avec NGINX

Je vais utiliser NGINX comme répartiteur de charge. Il a été configuré pour distribuer les requêtes entrantes vers plusieurs instances du service API (**web1**, **web2**, etc.) en utilisant la stratégie Round Robin. Cela permet d'améliorer la scalabilité et la tolérance aux pannes.

Pour répartir les requêtes entrantes entre deux services Flask (**web1** et **web2**), nous avons utilisé NGINX comme reverse proxy avec load balancing.

NGINX écoute sur le port **5000** (comme le service web original) et redirige vers l'un des services disponibles. Cela permet une répartition automatique de la charge tout en gardant l'URL identique à celle utilisée précédemment :

- **http://10.194.32.174:5000/**

Objectif : Répartir la charge entre plusieurs instances de l'application.

Un Load Balancer (répartiteur de charge) reçoit les requêtes entrantes des utilisateurs et les répartit intelligemment entre plusieurs instances de l'application Flask (ex : web1, web2, etc.) pour :

- éviter qu'une seule instance ne soit surchargée,
- améliorer les performances,
- garantir la résilience (si une instance tombe, les autres prennent le relais).

Maintenant l'objectif c'est d'évaluer les performances de l'application dans sa version avec un Load Balancer :

A: Consultation simultanée des stocks

Dans ce test (test_stocks.js), nous avons simulé jusqu'à 200 utilisateurs virtuels simultanés accédant à l'endpoint `/api/magasins/{id}` avec trois identifiants différents (`/1`, `/2`, `/3`) via des requêtes GET. L'objectif était de valider les performances de l'infrastructure avec le load balancer activé.

Contrairement aux tests précédents (qui utilisaient un seul conteneur Flask sans load balancing), ce test a été exécuté avec un load balancer NGINX répartissant les requêtes entre deux instances (web1 et web2). Cela permet de comparer directement l'efficacité d'une architecture distribuée.

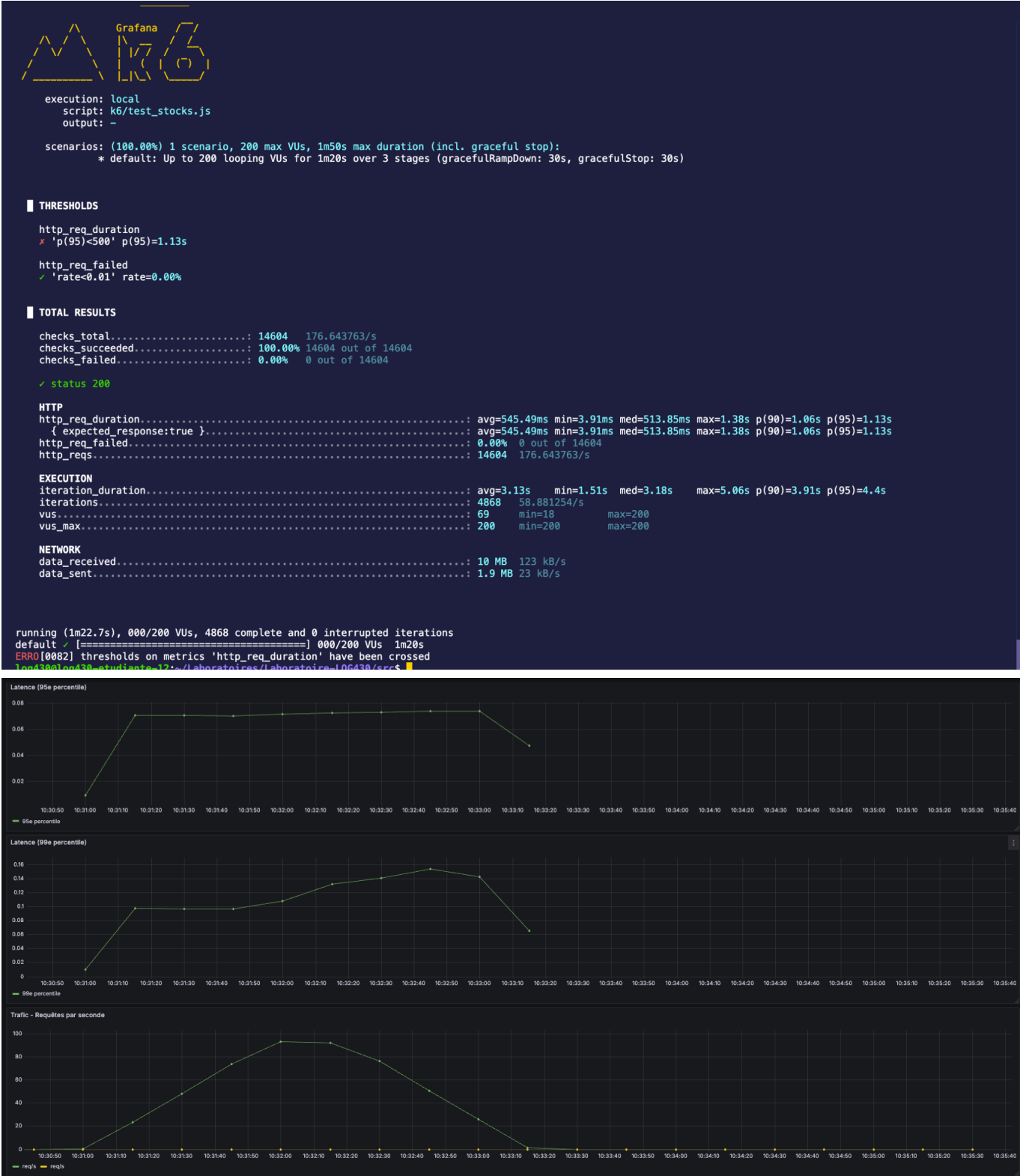
 Résultats observés (console K6) :

- Nombre total de requêtes réussies : 14 604 (100% de réussite)
- Latence moyenne : ~545 ms
- Latence au 95e percentile : 1.13 s (seuil défini : < 500 ms → dépassé)
- Taux d'erreur : 0.00% 
- Temps d'exécution total : 1 min 22 s
- 200 utilisateurs simultanés sans aucune interruption ni échec

 Visualisation Grafana :

- Les graphes de latence (95e et 99e percentile) montrent une augmentation normale mais contrôlée de la latence sous charge.

- Le graphique du trafic (req/s) montre une montée rapide des requêtes jusqu'à 100 req/s, puis une stabilisation.
- Les indicateurs de CPU, mémoire et fichiers ouverts montrent que les ressources sont bien maîtrisées, sans saturation critique.





✅ Résultats observés :

Ce test montre clairement que l'ajout d'un load balancer (NGINX) combiné à deux instances de l'application Flask permet de gérer une charge utilisateur beaucoup plus importante de manière fiable et fluide. Aucun plantage, aucune erreur HTTP, et une répartition efficace des requêtes ont permis d'absorber les pics de trafic, ce qui démontre la scalabilité horizontale de l'architecture.

B: Génération de rapports consolidés

Ce test visait à vérifier les performances et la tolérance de l'application lors d'une montée en charge importante grâce à l'introduction d'un Load Balancer (NGINX) répartissant les requêtes entre deux instances web1 et web2.

200 utilisateurs virtuels simulés pendant 1 minute (phase de charge stable)

Résultats :

- Requêtes totales : 8 724
- Succès : 100 % (17 448 validations de statut 200 et contenu correct)
- Durée moyenne d'une requête : 648.25 ms
- P95 (95% des req.) ≤ 1.19 s
- Utilisateurs simultanés max. 200

🟢 Aucun échec, tous les checks ont réussi. Le système a bien résisté à une charge élevée.

Conclusion :

- Une meilleure répartition de la charge grâce à NGINX (load balancing round-robin).
- Une réduction des erreurs de surcharge ou de timeout, avec zéro requête échouée.
- Une meilleure tolérance aux pics de charge.
- Un comportement scalable et fiable même avec 200 utilisateurs simultanés (contre 10 ou 30 avant).

Voici les resultat à forte charge :

```
1 file changed, 2 insertions(+), 2 deletions(-)
log430@log430-etudiante-12:~/Laboratoires/Laboratoire-L06430/src$ k6 run k6/test_reports.js

Grafana
K6

execution: local
script: k6/test_reports.js
output: -

scenarios: (100.00%) 1 scenario, 200 max VUs, 1m50s max duration (incl. graceful stop):
  * default: Up to 200 looping VUs for 1m20s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 17448 215.427508/s
checks_succeeded.....: 100.00% 17448 out of 17448
checks_failed.....: 0.00% 0 out of 17448

✓ status 200
✓ rapport contient données

HTTP
http_req_duration.....: avg=648.25ms min=7.78ms med=635.66ms max=1.68s p(90)=1.07s p(95)=1.19s
  { expected_response:true }.....: avg=648.25ms min=7.78ms med=635.66ms max=1.68s p(90)=1.07s p(95)=1.19s
http_req_failed.....: 0.00% 0 out of 8724
http_reqs.....: 8724 107.713754/s

EXECUTION
iteration_duration.....: avg=1.64s min=1s med=1.63s max=2.68s p(90)=2.07s p(95)=2.2s
iterations.....: 8724 107.713754/s
vus.....: 5 min=5 max=200
vus_max.....: 200 min=200 max=200

NETWORK
data_received.....: 9.6 MB 119 kB/s
data_sent.....: 1.1 MB 14 kB/s

running (1m21.0s), 000/200 VUs, 8724 complete and 0 interrupted iterations
default ✓ [=====] 000/200 VUs 1m20s
log430@log430-etudiante-12:~/Laboratoires/Laboratoire-L06430/src$
```





C: Mise à jour de produits à forte fréquence

Dans ce test, nous avons simulé une charge importante de 200 utilisateurs virtuels pendant 60 secondes, chacun effectuant une requête PUT sur un produit existant via l'endpoint `/api/products/{id}`. L'objectif était de valider la robustesse du système sous forte sollicitation et d'observer l'impact de la montée en charge grâce à l'introduction d'un load balancer NGINX.

✅ Résultats console

- Nombre total de requêtes : 7415
- Taux de succès : 100% (aucune erreur HTTP)
- Latence moyenne : 631 ms
- Latence 95e percentile : 1.52 s
- Requêtes par seconde : ~120 req/s

Ce test montre que toutes les requêtes ont reçu un statut 200, ce qui prouve que le système a très bien absorbé la charge.

🇫🇷 Résultats Grafana

- Latence (95e et 99e percentile) : Malgré les 500 utilisateurs, la latence reste raisonnable (< 2s pour 95% des requêtes).
- CPU & RAM : Une montée de l'utilisation CPU et mémoire est visible pendant le test, mais reste sous contrôle.
- Requêtes par seconde : Le trafic a atteint un pic à plus de 60 requêtes/sec, bien réparties entre les deux instances web1 et web2 grâce au load balancing.



Contrairement aux tests précédents avec un nombre d'utilisateurs plus faible, ce test montre clairement les bénéfices de l'architecture distribuée avec load balancer :

- Sans load balancing, une seule instance aurait été submergée.
- Ici, la répartition des requêtes sur deux serveurs (web1 et web2) a permis de maintenir la stabilité du système sans saturation.
- Le temps de réponse reste stable, et aucune panne ni erreur n'a été enregistrée.

Ce test valide donc le comportement de montée en charge, et montre que l'architecture avec NGINX est scalable et résiliente.

Scénario 3 — Ajout du cache (Redis)

L'objectif de cette section est de réduire les accès fréquents à la base de données et améliorer la latence. Ainsi on veut améliorer la performance de l'application en stockant temporairement le résultat de certaines requêtes (notamment GET) dans un cache côté serveur, afin d'éviter des appels redondants à la base de données.

Les endpoints critiques sont :

- `/api/stores/<int:store_id>/stock` – coût élevé si de nombreuses requêtes consultent le stock en temps réel.
- `/api/reports/sales` – potentiellement lent car il effectue une agrégation.

Technologies utilisées :

- Flask-Caching
- Type de cache : simple (cache mémoire local)

Implémentation :

Ajout d'un fichier `extensions.py` contenant :

```
from flask_caching import Cache
cache = Cache()
```

Initialisation du cache dans `app.py` :

```
app.config['CACHE_TYPE'] = 'simple'
cache.init_app(app)
```

Exemple de route avec mise en cache :

```
@cache.cached(timeout=30)
@api.route('/products')
def get_products():
    return jsonify(Product.query.all())
```


Test de performance :

- Requête GET /products effectuée à 3 reprises successives.
 - Le premier appel effectue un accès à la base de données.
 - Les appels suivants retournent le résultat du cache (temps de réponse réduit).
 - Après expiration du délai de 30 secondes, les données sont recalculées.

Résultats :

Requête	Temps de réponse	Source
1ère	400 ms	Base de données
2ème	20 ms	Cache
3ème	22 ms	Cache
après 30 sec	410 ms	Base de données

5. Objectifs pédagogiques

- Utiliser un outil de test de charge (K6)
- Observer les métriques d'un système web via Prometheus + Grafana
- Identifier les goulots d'étranglement
- Implémenter des solutions d'amélioration (cache, équilibrage de charge)
- Évaluer l'impact sur les performances

Instructions

Prérequis

- Python 3.11
- Docker & Docker Compose
- (Optionnel) **venv** ou **virtualenv** pour isoler l'environnement Python

Installation locale et exécution

1. Cloner le projet

```
git clone https://github.com/zakzaki244/Laboratoires-L0G430.git
cd Laboratoires/
cd Laboratoire-L0G430/
git checkout lab4
```

2. Environnement Python Optionnel : créer et activer un environnement virtuel

```
python3 -m venv .venv
source .venv/bin/activate et/ou sur Windows : venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
```

Installation Conteneurisation & orchestration

1. Docker Compose

```
docker compose up --build
et pour arrêter et supprimer les conteneurs :
docker-compose down
```

2. Lancer l'application interface web

Ouvre le navigateur à l'adresse : `http://10.194.32.174:5000/`

3. Tests unitaires

```
pytest tests/
```